

# CONVENTION MAÎTRE DU PROJET

## Plateforme SaaS intelligente pour enseignants

Version : 2.0 — Document de référence

---

## 1. OBJET DE LA CONVENTION

Cette Convention définit les règles fondamentales qui doivent gouverner la conception, le développement, la sécurité, l'évolution et l'utilisation de la plateforme.

Elle constitue la **source de vérité du projet**.

Tout développement futur doit respecter :

1. cette Convention ;
2. le PRD ;
3. le cahier des fonctionnalités ;
4. le modèle de données ;
5. les règles officielles des systèmes éducatifs intégrés.

En cas de conflit, la cohérence de l'architecture et la sécurité des données sont prioritaires, puis les règles métier validées du produit.

---

## 2. VISION DU PRODUIT

La plateforme est un **SaaS professionnel destiné aux enseignants**, conçu en priorité pour les systèmes scolaires africains.

Le premier marché est :

**CI Côte d'Ivoire**

L'architecture doit cependant permettre d'intégrer progressivement d'autres pays africains **sans reconstruire le produit**.

La plateforme n'est **pas une plateforme de cours en ligne**.

Elle accompagne le professeur dans son travail **réel en présentiel** :

- préparation ;
  - planification ;
  - enseignement ;
  - évaluations ;
  - correction ;
  - suivi des élèves ;
  - analyse des performances ;
  - remédiation ;
  - progression pédagogique.
- 

### 3. POSITIONNEMENT

La plateforme ne doit pas être conçue comme :

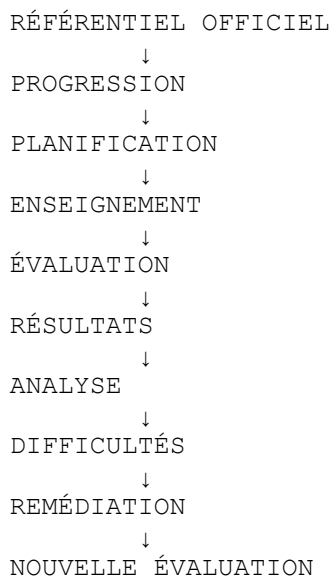
« un générateur de cours avec IA ».

Elle doit être conçue comme :

**un environnement de travail intelligent pour le professeur.**

L'IA est un composant du produit, et non le produit entier.

Le cœur du produit est la chaîne :



### 4. PRINCIPLE « TEACHER FIRST »

Chaque fonctionnalité doit être pensée d'abord pour le professeur.

L'application doit :

- réduire son travail administratif ;
- réduire le temps de préparation ;
- faciliter la création de contenus ;
- faciliter les évaluations ;
- faciliter la correction ;
- rendre les données compréhensibles ;
- l'aider à identifier les difficultés ;
- lui proposer des actions concrètes.

L'application ne doit jamais ajouter de complexité sans bénéfice clair.

---

## 5. LE PROFESSEUR RESTE DÉCISIONNAIRE

L'IA assiste le professeur.

Elle ne remplace pas son jugement professionnel.

Pour les actions importantes :

```
PROPOSITION
↓
VÉRIFICATION
↓
MODIFICATION POSSIBLE
↓
VALIDATION DU PROFESSEUR
```

Exemples :

- cours généré ;
  - devoir ;
  - interrogation ;
  - corrigé ;
  - appréciation ;
  - remédiation ;
  - analyse importante.
- 

## 6. RÈGLE DE VÉRACITÉ

La plateforme ne doit jamais présenter comme officiel ce qui ne l'est pas.

Trois catégories doivent être clairement distinguées :

### A. Contenu officiel

Issu d'un référentiel ou document reconnu.

### B. Contenu du professeur

Créé ou importé par l'enseignant.

### C. Contenu généré

Produit par l'IA ou le système.

Cette distinction doit être conservée dans les données et, lorsque nécessaire, visible dans l'interface.

---

## 7. SYSTÈME ÉDUCATIF

Le système scolaire doit être modélisé comme une donnée structurée.

```
Pays
↓
Système éducatif
↓
Année scolaire
↓
Cycle
↓
Niveau
↓
Matière
↓
Programme
↓
Chapitre / notion
↓
Compétence
↓
Progression officielle
```

Aucune partie du produit ne doit être construite en supposant que tous les pays utilisent la même organisation scolaire.

---

## 8. CÔTE D'IVOIRE — PREMIER RÉFÉRENTIEL

La Côte d'Ivoire constitue le premier système éducatif intégré.

Les programmes et progressions ivoiriens doivent être stockés dans le système de manière structurée et versionnée.

La plateforme doit pouvoir distinguer :

- pays ;
  - année scolaire ;
  - niveau ;
  - matière ;
  - programme ;
  - progression ;
  - version du référentiel.
- 

## 9. AJOUT DE NOUVEAUX PAYS

L'ajout d'un nouveau pays ne doit pas nécessiter de modifier le cœur du logiciel.

Il doit principalement consister à ajouter :

```
COUNTRY  
EDUCATION SYSTEM  
LEVELS  
SUBJECTS  
CURRICULA  
COMPETENCIES  
OFFICIAL PROGRESSIONS  
OFFICIAL DOCUMENTS
```

Il est interdit de créer une architecture spécifique du type :

```
senegal_tables  
benin_tables  
togo_tables
```

Le modèle doit rester générique.

---

## 10. VERSIONNAGE DES PROGRAMMES

Un programme officiel ne doit jamais être simplement écrasé par une nouvelle version.

Une nouvelle version doit être identifiable.

Exemple :

Côte d'Ivoire  
Mathématiques  
3e  
Programme  
Version 2026

puis :

Côte d'Ivoire  
Mathématiques  
3e  
Programme  
Version 2027

L'historique doit rester accessible selon les permissions appropriées.

---

## 11. PROGRESSION OFFICIELLE

La progression officielle est une référence.

La progression réelle du professeur est une autre donnée.

Elles ne doivent jamais être fusionnées.

Le système doit pouvoir comparer :

OFFICIEL  
VS  
RÉEL

---

## 12. ANALYSE DE PROGRESSION

Le système peut détecter :

- retard ;
- avance ;
- conformité ;
- élément non traité ;
- données insuffisantes.

Mais une alerte doit être accompagnée d'éléments permettant au professeur de comprendre **pourquoi** elle apparaît.

---

## 13. EMPLOI DU TEMPS

Le professeur doit pouvoir :

- créer son emploi du temps ;
- modifier ses créneaux ;
- consulter son planning ;
- importer une photo ;
- importer un fichier ;
- vérifier les informations extraites.

L'application peut utiliser l'emploi du temps pour aider à planifier les séances.

---

## 14. PHOTOS ET DOCUMENTS

Le professeur peut importer :

- emploi du temps ;
- fiche de notes ;
- liste d'élèves ;
- cours ;
- fiches pédagogiques ;
- documents administratifs ;
- autres fichiers pédagogiques.

Les formats pris en charge peuvent évoluer.

---

## 15. ORIGINALITÉ DES DOCUMENTS

**L'original ne doit jamais être détruit ou remplacé automatiquement.**

Architecture :

```
DOCUMENT ORIGINAL
  ↓
EXTRACTION
  ↓
DONNÉES EXTRAITES
  ↓
VÉRIFICATION
```

↓  
IMPORT DANS LE SYSTÈME

Une erreur OCR doit pouvoir être corrigée sans modifier le fichier original.

---

## 16. OCR ET IA DOCUMENTAIRE

Lorsqu'une photo est analysée, le système doit pouvoir fournir :

- données extraites ;
- niveau de confiance lorsque disponible ;
- éléments nécessitant vérification.

L'utilisateur doit pouvoir corriger les erreurs avant validation.

---

## 17. GÉNÉRATION DE CONTENU

L'IA peut générer :

- cours ;
  - exercices ;
  - devoirs ;
  - interrogations ;
  - contrôles ;
  - compositions ;
  - corrigés ;
  - remédiations ;
  - appréciations ;
  - recommandations.
- 

## 18. CONTEXTE OBLIGATOIRE DE L'IA

Lorsque les informations sont disponibles, l'IA doit tenir compte du contexte :

Pays  
Année scolaire  
Niveau  
Classe  
Matière  
Programme

Chapitre  
Compétence  
Progression  
Durée disponible  
Objectif

Elle ne doit pas générer un contenu générique lorsque le contexte pédagogique nécessaire est connu.

---

## 19. IA ET PROGRAMMES OFFICIELS

L'IA ne doit jamais inventer une règle en la présentant comme une exigence officielle.

Si une information officielle n'est pas disponible :

le système doit l'indiquer.

Il ne doit pas fabriquer une « progression officielle ».

---

## 20. ANALYSE DES ÉLÈVES

Le système doit pouvoir identifier des tendances à partir des données disponibles.

Exemple :

Un élève présente des difficultés persistantes sur plusieurs évaluations associées à une notion.

Le système peut alors signaler :

- élève concerné ;
  - matière ;
  - chapitre ;
  - compétence ;
  - indicateurs ayant conduit à l'alerte.
- 

## 21. PAS DE JUGEMENT AUTOMATIQUE SUR L'ÉLÈVE

La plateforme doit éviter les formulations définitives telles que :

« Cet élève est mauvais en mathématiques. »

Elle doit privilégier :

« Les résultats disponibles indiquent une difficulté persistante sur les équations. »

L'objectif est d'identifier **une difficulté pédagogique**, pas d'étiqueter l'élève.

---

## 22. ANALYSE DES CHAPITRES

Le système doit pouvoir identifier les chapitres ou notions présentant des difficultés collectives.

Exemple :

Classe : 3e A

Fonctions

Moyenne : 8,2/20

Élèves concernés : 11

Puis proposer :

**Créer une séance de remédiation**

---

## 23. REMÉDIATION

Une remédiation peut être :

- individuelle ;
- en petit groupe ;
- collective.

Elle doit pouvoir contenir :

- objectif ;
  - prérequis ;
  - rappel ;
  - explications ;
  - exercices ;
  - correction ;
  - évaluation courte.
-

## 24. TRAÇABILITÉ DE L'ANALYSE

Lorsqu'une recommandation est générée, le système doit pouvoir conserver les éléments utilisés pour la produire.

Exemple :

```
Recommandation
  ↓
Résultats utilisés
  ↓
Évaluations
  ↓
Questions
  ↓
Compétences / notions
```

Cela améliore la transparence et la confiance.

---

## 25. MODÈLE MULTI-TENANT

Le produit est multi-tenant.

Une organisation ne doit jamais pouvoir accéder aux données d'une autre organisation.

Structure :

```
ORGANIZATION
  ↓
ESTABLISHMENT
  ↓
TEACHERS
CLASSES
DOCUMENTS
DATA
```

---

## 26. ISOLATION DES DONNÉES

Les données doivent être protégées au niveau de la base de données.

La sécurité ne doit pas reposer uniquement sur l'interface.

---

## 27. RLS OBLIGATOIRE

Les tables contenant des données privées doivent utiliser **Row Level Security**.

Cela concerne notamment :

- élèves ;
  - notes ;
  - classes ;
  - documents ;
  - évaluations ;
  - résultats ;
  - analyses ;
  - abonnements ;
  - données d'organisation.
- 

## 28. RÔLES

Les rôles sont séparés des profils.

Rôles possibles :

OWNER  
ADMIN  
MANAGER  
TEACHER  
READER

Un utilisateur peut avoir des permissions différentes selon l'organisation.

---

## 29. PERMISSIONS

Une personne ne doit accéder qu'aux données nécessaires à son rôle.

Exemple :

Un professeur peut modifier ses données pédagogiques.

Un administrateur d'établissement peut gérer les enseignants selon les permissions définies.

Un lecteur peut consulter certaines données sans pouvoir les modifier.

---

## 30. AUDIT

Les opérations sensibles doivent être journalisées.

Exemples :

- modification d'une note ;
  - suppression ;
  - changement de rôle ;
  - changement de permission ;
  - action administrative importante.
- 

## 31. NOTES

Les notes doivent être structurées et validées.

Le système doit empêcher notamment :

- note supérieure au maximum ;
  - valeur invalide ;
  - élève inexistant ;
  - doublon accidentel.
- 

## 32. ARGENT

Les montants financiers doivent être stockés sous forme d'entiers dans la plus petite unité monétaire.

**Jamais de `float` pour les montants financiers.**

---

## 33. ABONNEMENTS

Deux grands modèles sont prévus :

## Professeur individuel

Le professeur paie son propre abonnement.

## Établissement

L'établissement achète des licences pour ses enseignants.

Le système doit donc gérer :

```
Organization
  ↓
Subscription
  ↓
Plan
  ↓
Licenses
  ↓
Teachers
```

---

## 34. ARCHITECTURE DU MODÈLE DE DONNÉES

Le modèle doit rester normalisé et relationnel.

Les relations principales sont :

```
COUNTRY
  ↓
EDUCATION SYSTEM
  ↓
CURRICULUM
  ↓
CURRICULUM UNITS
  ↓
COMPETENCIES
```

et :

```
ORGANIZATION
  ↓
ESTABLISHMENT
  ↓
CLASS
  ↓
STUDENT
```

et :

```
TEACHER
  ↓
LESSON
  ↓
```

ASSESSMENT  
↓  
RESULT  
↓  
ANALYSIS  
↓  
REMEDIATION

---

## 35. PAS DE DUPLICATION INUTILE

Une même information ne doit pas être copiée dans plusieurs tables sans raison.

Utiliser :

- clés étrangères ;
  - relations ;
  - tables d'association ;
  - référentiels centralisés.
- 

## 36. HISTORIQUE

Les données importantes doivent conserver leur historique lorsque nécessaire.

Exemples :

- changement de classe ;
  - année scolaire ;
  - programme ;
  - progression ;
  - résultats ;
  - remédiation.
- 

## 37. ANNÉE SCOLAIRE

Les données pédagogiques doivent être correctement rattachées à l'année scolaire.

Le passage à une nouvelle année ne doit pas supprimer l'historique.

---

## 38. DESIGN UI/UX

L'interface doit être :

- premium ;
- claire ;
- moderne ;
- professionnelle ;
- responsive ;
- accessible.

Elle doit fonctionner sur :

- ordinateur ;
  - tablette ;
  - smartphone.
- 

## 39. PRIORITÉ MOBILE

Certaines actions doivent être particulièrement simples sur mobile :

- prendre une photo ;
  - importer un document ;
  - consulter l'emploi du temps ;
  - consulter une alerte ;
  - saisir des notes ;
  - consulter une fiche élève.
- 

## 40. DASHBOARD

Le dashboard doit répondre immédiatement à :

**Que dois-je faire aujourd'hui ?**

et :

**Y a-t-il quelque chose qui nécessite mon attention ?**

Il ne doit pas être uniquement composé de graphiques.

---

## 41. RECHERCHE GLOBALE

La recherche doit permettre de retrouver rapidement :

- élèves ;
  - classes ;
  - cours ;
  - exercices ;
  - évaluations ;
  - documents ;
  - chapitres ;
  - résultats.
- 

## 42. ACCESSIBILITÉ

Prévoir notamment :

- contraste suffisant ;
  - navigation clavier ;
  - focus visible ;
  - labels ;
  - messages d'erreur compréhensibles ;
  - respect de `prefers-reduced-motion`.
- 

## 43. INTERNATIONALISATION

L'application doit être conçue pour plusieurs langues.

La langue doit pouvoir être déterminée par :

1. préférence utilisateur ;
2. navigateur ;
3. langue par défaut de l'établissement ;
4. fallback.

Le français est prioritaire au lancement.

---

## 44. ARCHITECTURE TECHNIQUE

Le produit doit rester compatible avec l'architecture retenue pour le projet.

La couche frontend, backend, base de données, authentification, stockage et fonctions serveur doivent être clairement séparées.

Aucune fonctionnalité critique ne doit dépendre exclusivement du frontend pour sa sécurité.

---

## 45. SUPABASE

La base doit utiliser :

- PostgreSQL ;
- Auth ;
- Storage ;
- RLS ;
- fonctions serveur lorsque nécessaire.

La base de données est considérée comme une composante centrale du produit.

---

## 46. STOCKAGE

Les fichiers doivent être stockés dans un système de stockage sécurisé.

Les URLs publiques permanentes ne doivent pas être utilisées pour des documents privés contenant des données d'élèves.

---

## 47. DONNÉES SENSIBLES

Les données concernant les élèves doivent être traitées comme des données nécessitant une protection renforcée.

Le système doit appliquer :

- minimisation ;
- contrôle d'accès ;
- isolation ;

- audit ;
  - stockage sécurisé.
- 

## 48. SUPPRESSION

Une suppression importante ne doit pas entraîner une perte accidentelle d'informations historiques essentielles.

Lorsque cela est approprié :

archivage plutôt que suppression définitive.

---

## 49. VERSIONNAGE DES CONTENUS

Les contenus générés ou modifiés doivent pouvoir être distingués lorsqu'une évolution importante intervient.

Exemple :

```
Cours v1  
Cours v2  
Cours v3
```

Cela facilite la récupération d'une version antérieure.

---

## 50. IA — INTERDICTION DES ACTIONS SILENCIEUSES

L'IA ne doit pas :

- modifier des notes silencieusement ;
  - modifier un programme officiel ;
  - supprimer des élèves ;
  - changer des permissions ;
  - publier automatiquement une évaluation importante ;
  - modifier l'emploi du temps sans validation.
-

## 51. IA — EXPLICATION

Lorsqu'une recommandation importante est faite, l'interface doit privilégier :

**Pourquoi cette recommandation ?**

plutôt que simplement :

**Voici la recommandation.**

---

## 52. GÉNÉRATION DE CONTENU

Le professeur doit pouvoir :

- accepter ;
  - modifier ;
  - régénérer ;
  - refuser ;
  - sauvegarder.
- 

## 53. QUALITÉ DES CONTENUS

Avant publication ou utilisation, les contenus générés doivent être vérifiables.

Une génération IA n'est pas considérée automatiquement comme correcte.

---

## 54. DONNÉES MANQUANTES

Lorsque le système ne dispose pas de suffisamment d'informations, il doit le dire.

Exemple :

« Analyse impossible : aucune note n'est disponible pour cette période. »

Il ne doit pas inventer une conclusion.

---

## 55. CONFLITS DE DONNÉES

En cas de données contradictoires :

DONNÉE A  
≠  
DONNÉE B

le système doit signaler le conflit et demander une résolution lorsque nécessaire.

---

## 56. IMPORTATIONS

Toute importation doit suivre :

IMPORT  
↓  
ANALYSE  
↓  
VALIDATION  
↓  
INTÉGRATION

Pas :

IMPORT  
↓  
INSERTION AUTOMATIQUE

pour les données sensibles ou potentiellement ambiguës.

---

## 57. TESTS

Chaque fonctionnalité importante doit être testée :

Fonctionnel

Est-ce que cela fonctionne ?

Sécurité

Un utilisateur non autorisé peut-il y accéder ?

## Données

Les relations sont-elles correctes ?

## Interface

Le parcours est-il compréhensible ?

## Régression

La nouvelle fonctionnalité casse-t-elle quelque chose ?

---

# 58. RÈGLE DE NON-RÉGRESSION

Une nouvelle fonctionnalité ne doit pas casser une fonctionnalité existante.

Avant une modification importante :

1. identifier les dépendances ;
  2. modifier ;
  3. tester ;
  4. vérifier les données ;
  5. vérifier les permissions ;
  6. vérifier l'interface.
- 

# 59. PAS DE FONCTIONNALITÉS INVENTÉES

L'IA de développement ne doit pas ajouter arbitrairement des fonctionnalités parce qu'elle estime qu'elles seraient intéressantes.

Toute nouvelle fonctionnalité doit être :

- demandée ;
  - validée ;
  - ou explicitement proposée puis approuvée.
-

## 60. MVP

Le MVP doit rester concentré.

Priorité :

PROFESSEUR  
↓  
CLASSE  
↓  
ÉLÈVES  
↓  
PROGRAMME  
↓  
PROGRESSION  
↓  
COURS  
↓  
ÉVALUATION  
↓  
NOTES  
↓  
ANALYSE  
↓  
REMÉDIATION

Les fonctionnalités secondaires peuvent attendre.

---

## 61. ÉVOLUTION FUTURE

L'architecture doit permettre d'ajouter progressivement :

- présence ;
  - cahier de texte avancé ;
  - bulletins ;
  - portail établissement ;
  - portail parents ;
  - portail élèves ;
  - nouveaux pays ;
  - application mobile native ;
  - analytics avancés ;
  - nouvelles intégrations de paiement.
-

## 62. RÈGLE D'EXTENSIBILITÉ

Une nouvelle fonctionnalité doit être ajoutée sans casser :

- les données existantes ;
  - les relations ;
  - les permissions ;
  - les référentiels ;
  - les abonnements.
- 

## 63. SOURCE DE VÉRITÉ

Les documents suivants constituent la documentation officielle du projet :

```
01 - CONVENTION MAÎTRE
      ↓
02 - PRD
      ↓
03 - CAHIER DES FONCTIONNALITÉS
      ↓
04 - MODÈLE DE DONNÉES
      ↓
05 - SCHÉMA SQL / SUPABASE
      ↓
06 - ARCHITECTURE TECHNIQUE
      ↓
07 - DESIGN SYSTEM
```

Chaque niveau doit respecter celui qui le précède.

---

## 64. RÈGLE FINALE

Avant toute implémentation importante, la question doit être :

**« Est-ce conforme à la Convention ? »**

Si la réponse est non :

**on ne construit pas la fonctionnalité avant d'avoir validé la modification de la Convention.**

---

## 65. FORMULE DIRECTRICE DU PROJET

La plateforme doit permettre au professeur de passer de :

**« Je dois préparer mon cours »**

à :

**« Je sais quoi enseigner, pourquoi je l'enseigne, où j'en suis dans le programme, comment l'enseigner, comment évaluer mes élèves, quels élèves ont besoin d'aide, sur quoi ils ont des difficultés et quelle action pédagogique je peux mettre en place. »**

C'est cette promesse qui doit guider tout le produit.